

A Real-Time Juturna Pipeline for Live Audio Summarization and Keyword Extraction

Mervegul Parlak
Ozyegin University
Istanbul, Turkiye

Burak Kara
Rennes, France

Alperen F. Zengin,
Ali C. Begen
Ozyegin University
Istanbul, Turkiye

Abstract—Streaming summarization of live audio requires systems that maintain coherence across overlapping speech segments while satisfying strict per-window latency constraints. We address the Streaming Transcription and Real-Time Extraction for AI Meeting Intelligence Distillation (STREAMIND) Grand Challenge with an eight-node Juturna pipeline in which live Opus/RTP audio is ingested via the Janus WebRTC gateway, transcribed incrementally by faster-whisper, and summarized by Qwen3.5-4B-AWQ under vLLM with a 300 s rolling context window. Three design contributions target the challenge composite scoring. Streaming chunk deduplication removes overlap-induced redundancy to protect summary quality; a keyword guarantee algorithm enforces the three-keyword format and back-fills relevant terms from the transcript; and prompt engineering with thinking-mode suppression holds average processing time to 0.79 s. Across Rev16 podcast episodes, the pipeline achieves a cross-episode mean source score of 40.14 ± 0.85 out of 45, with consistent scores on technical meeting sessions indicating generalization beyond the podcast domain.

Index Terms—real-time speech processing, automatic speech recognition, large language models, streaming summarization, WebRTC, Juturna

I. INTRODUCTION

Live-audio summarization of meetings, conference sessions, and podcasts requires incremental outputs under strict latency constraints while maintaining coherence across overlapping speech segments. The Streaming Transcription and Real-Time Extraction for AI Meeting Intelligence Distillation (STREAMIND) Grand Challenge¹ formalizes this setting, requiring a system to process a live RTP audio stream and emit a structured summary with three keywords every 300 s window, scored jointly on summary quality, keyword relevance, and processing latency.

The challenge scores the output of each window i as $C_i = B_i + K_i + L_i$, where $B_i \in [5, 25]$ measures summary quality across five Likert criteria, $K_i \in [-6, +6]$ rewards relevant keywords and penalizes irrelevant ones, and the latency bonus $L_i = 10 \cdot e^{-0.5 \cdot \text{proc}_i}$ rewards faster processing. When $B_i < 10$, this bonus is withheld regardless of processing speed. Fig. 1 illustrates the L_i score under changing processing times. Moreover, per challenge description, using the Janus WebRTC gateway [1] as the audio source adds a flat +4 bonus to the overall final score. This scoring structure dictates a clear design priority, namely to drive B_i above the gate first,

minimize proc_i at fixed quality, and treat any K_i regression as unacceptable since one irrelevant keyword erases multiple points of latency gain.

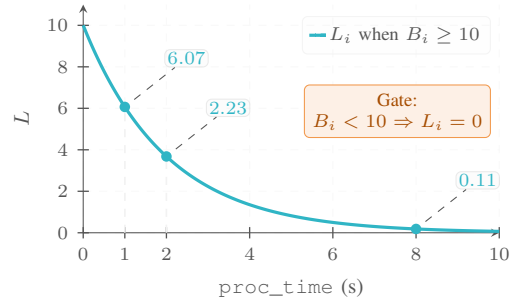


Fig. 1: Latency bonus $L_i = 10 e^{-0.5 \cdot \text{proc}_i}$, gated by $B_i \geq 10$. Below the gate, latency contributes zero; above it, returns diminish as processing time increases.

To address the STREAMIND Grand Challenge, we build a real-time multimedia pipeline and present three contributions, each targeting a specific scoring term:

- 1) A streaming chunk deduplication strategy that preserves temporal coherence across overlapping 5 s Automatic Speech Recognition (ASR) chunks while keeping the aggregated 300 s window non-redundant (B_i).
- 2) A keyword guarantee algorithm that strips banned generic terms and back-fills missing slots from transcript-derived terms, enforcing the challenge three-keyword requirement so that format failures never forfeit the keyword score K_i .
- 3) Quality-gated latency optimization through prompt engineering and thinking-mode suppression, maintaining B_i above 23 of 25 while achieving 0.79 s average processing time ($L_i \approx 6.8$).

The remainder of this paper is organized as follows. Section II surveys ASR and Large Language Model (LLM) inference options and situates the work among prior streaming meeting summarization pipelines. Section III details the eight pipeline nodes and their role. Section IV covers inference-engine selection and submission configuration. Section V reports ASR model comparison, per-window scoring, component ablations, and the model-size and cross-family sweep. Section VI discusses scoring trade-offs, summarizes contributions, and outlines future directions.

¹<https://attend.ieee.org/mmmsp-2026/grand-challenges/>

II. BACKGROUND AND RELATED WORK

Building a live audio summarization system for the STREAM-MIND Grand Challenge draws on three components: the Juturna [2] pipeline framework, the optional Janus [1] WebRTC gateway, and LLM inference for summarization and keyword extraction. Juturna [2] is an open-source Python library for composing real-time multimedia pipelines from typed, message-passing nodes arranged as a directed acyclic graph in JSON. Each node is driven through a four-stage lifecycle (`configure` $\rightarrow$ `warmup` $\rightarrow$ `start` $\rightarrow$ `stop`), and internally separates reception, buffering, and processing into independent threads, which enables stages to absorb backpressure without stalling upstream data flow. Nodes take one of three roles, *source* (data ingestion), *processing* (transform), or *sink* (delivery), and exchange strongly-typed payloads along directed edges.

Janus [1] is an open-source, general-purpose WebRTC server that handles session negotiation and forwards decoded media as plain RTP to downstream consumers. Built-in `audio_rtp` source in Juturna integrates with the JanusRTC endpoint, receiving mono Opus/48 kHz audio [3], [4].

To transcribe the incoming audio, the pipeline builds on the Whisper model family [5], which achieves broad multilingual coverage through weak supervision on large-scale web audio. CTranslate2-based [6] reimplementations such as `faster-whisper` [7] reduce inference latency relative to the reference implementation while preserving accuracy at `float16` precision on the GPU.

The resulting transcripts are summarized by instruction-tuned LLMs, which can produce structured summaries and keyword lists in a single inference call [8]. Several inference backends serve such models with different trade-offs. vLLM [9] pairs paged attention and continuous batching with an in-process LLM class that needs no separate inference daemon and installs from mature prebuilt wheels. SGLang [10] reports around 29% higher throughput through RadixAttention prefix caching, but that advantage targets high-concurrency, shared-prefix serving rather than a single live stream, and its serving stack carries heavier build dependencies. TensorRT-LLM [11] maximizes raw throughput through per-model ahead-of-time compilation, which impedes rapid model iteration, whereas llama.cpp [12] targets portable CPU inference via GGUF quantization rather than the GPU. For a single-stream, rapidly-iterated, offline-packaged challenge entry, the vLLM zero-compilation install and self-contained execution outweigh the SGLang throughput edge: on the latency curve (Fig. 1), even a full 29% latency translation lifts L_i by under one point at sub-second processing.

Offline evaluation of LLM-generated summaries follows the LLM-as-judge paradigm [13], scoring outputs against source transcripts on multiple quality criteria, with judges drawn from distinct model families to reduce single-model bias.

The components above situate within a body of prior work on real-time ASR and meeting summarization that informs three aspects of our design. Streaming transcription

approaches include Whisper adapted via a local agreement policy [14] and dialect-specific pipelines that couple ASR with direct summarization [15]. Our pipeline instead uses fixed overlapping chunks with greedy suffix-prefix deduplication rather than adaptive windowing. On pipeline structure, cascade ASR+LLM systems have been validated sentence-wise at Interspeech [16] and integrated with extractive NLP as unified audio-to-summary applications [17]. Liu and Chen [18] apply a dynamic sliding window to long meeting transcripts. The challenge specification mandates the same 300s window granularity, and we extend it with in-process LLM inference in place of extractive NLP. Asthana *et al.* [19] validate structured LLM output for post-meeting recap, informing our prompt design. Schneider *et al.* [20] formalize quality-latency trade-offs for online summarization. Our system embeds these trade-offs as the B_i -gated L_i scoring term per challenge rather than an abstract policy objective. Collectively, these designs establish the building blocks our pipeline assembles into a scoring-formula-aware, end-to-end live challenge system.

III. SYSTEM ARCHITECTURE

Figure 2 shows the eight-node pipeline; each node targets one or more terms of $C_i = B_i + K_i + L_i$. Janus provides WebRTC audio ingestion; the hallucination filter and novel extractor deliver a clean, non-redundant transcript that protects B_i ; keyword validation enforces the three-keyword format to avoid penalty in K_i ; and the summarizer balances output quality against processing time at the B_i/L_i interface.

A. Audio Reception

Janus [1] handles the challenge WebRTC session and forwards mono Opus/48 kHz RTP packets to the Juturna `audio_rtp` source node on UDP port 8888, matching the challenge stream specification.

B. Incremental ASR

The `audio_chunker` node buffers the continuous audio stream into 5s chunks with 1s overlap, yielding a 4s advance per step. Each chunk is transcribed by the `transcriber_whisper` node running the ASR model `faster-whisper-large-v3-turbo` [21] at `float16` precision on the GPU, occupying around 2 GB of VRAM. The distilled model along with the summarizer model fit within the 24 GB development card with headroom for the Key-Value (KV) cache. Previous-text conditioning is disabled to prevent hallucination propagation across chunks, and Voice Activity Detection (VAD) filtering via Silero [22] skips silence segments.

C. Hallucination Filter

The `hallucination_filter` node applies two post-ASR guards on the transcription. First, inline noise tags (*e.g.*, [Music], (applause)) are stripped via regex patterns. Second, the text is checked against 22 known Whisper hallucination patterns (*e.g.*, “please subscribe”, “like and subscribe”) and discarded if it matches one. Without this filter, hallucinated sentences reach the summarizer and are reproduced as facts in

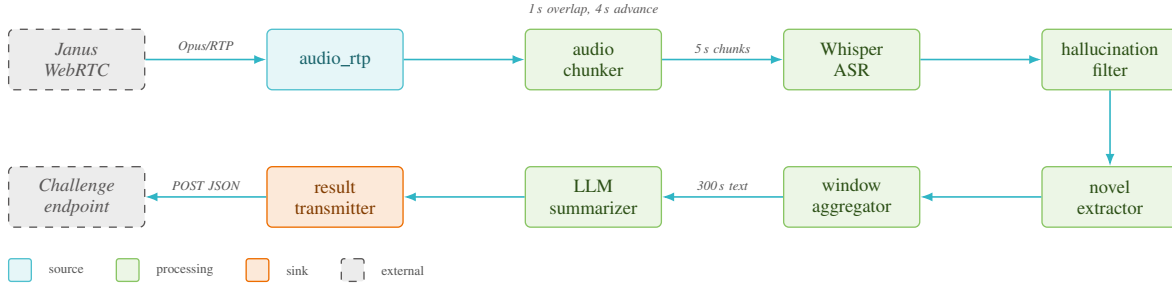


Fig. 2: End-to-end STREAMIND pipeline. Janus forwards Opus/RTP media into the Juturna `audio_rtp` source node; 5s chunks with a 4s advance are transcribed by ASR; hallucination filtering and novel extraction precede a 300s aggregation buffer; the summarizer emits output per window; the result sink stores locally and posts to the challenge endpoint.

the generated summary, inflating context token usage, increasing processing latency, and penalizing the factual consistency component of B_i .

D. Novel Chunk Extraction

The 1s overlap between consecutive chunks means that this interval of speech appears in both the current and previous transcription. The `novel_extractor` node performs word-level overlap detection by greedily testing suffix-prefix matches from length 10 words down to 1 and accepting the first exact match found. An exact match returns the overlap length; a `SequenceMatcher` fallback with similarity threshold 0.8 handles minor ASR variations in the overlap region (e.g., “the quick brown” vs. “the quicker brown”). Only the novel words beyond the detected overlap are emitted. Without this deduplication, the 300s window would contain around 20% redundant text, wasting context budget and biasing summaries toward repeated phrases.

E. Window Aggregation

The `window_aggregator` node concatenates incoming novel segments into a rolling 300s context buffer, matching the challenge scoring granularity. A window is flushed and forwarded to the summarizer when $t_{\text{chunk_end}} \geq t_{\text{window_start}} + 300$. On pipeline shutdown, any partial window accumulated since the last flush is emitted as well, ensuring the final audio segment contributes a scored output rather than being dropped.

F. Summarization

At warmup, the `summarizer_vllm` node loads model weights from a local filesystem path through the vLLM in-process LLM class [9], eliminating any inference daemon or runtime network dependency. Paged attention keeps proc_i low across variable transcript lengths, a direct lever on L_i , and the explicit `gpu_memory_utilization` budget lets the summarizer co-reside with the ASR model on the development GPU. A dummy generation call at startup triggers CUDA kernel compilation before the first live window, removing the cold-start spike that would otherwise inflate proc_i .

We utilize Qwen3.5-4B [8], [23] quantized to `int4` via Activation-aware Weight Quantization (AWQ) [24] at 4096 context length. AWQ quantization roughly quarters the weight

footprint relative to `bfloat16`, reducing inference latency and preserving KV cache headroom for 300s windows.

The summarization prompt targets the five Likert evaluation criteria. *2–3 sentences capturing the main topics and conclusions* targets conciseness and coherence. *Only state facts explicitly present in the transcript. Do not infer* targets factual consistency. *Exactly 3 keywords [...]* targets keyword relevance (K_i). Chain-of-thought generation is suppressed via `enable_thinking=False`, reducing output token count and thus proc_i . Sampling parameters are set conservatively, with temperature 0.15 and `top-p = 0.9` promoting deterministic, on-topic outputs, repetition penalty 1.05 discouraging verbatim transcript echoing, and `max_tokens = 384` capping the generation length.

Two failure modes are handled to ensure the pipeline never emits an empty summary. When the transcript is shorter than 80 characters (e.g., silence), the LLM is skipped. When the call raises an exception, returns unparseable JSON, or produces an empty summary, the pipeline falls back to an *extractive summary* consisting of the first sentence(s) of the transcript, capped at 300 characters. This verbatim excerpt guarantees high factual consistency (B_i cannot be penalized for a direct quote) and costs around 0 ms, preserving L_i .

G. Keyword Validation

Regardless of LLM output quality, the `ensure_three_keywords` function guarantees three keywords per window. The algorithm first removes empty entries and filters out banned generic terms (“meeting”, “discussion”, “topic”, etc.). Case-insensitive deduplication preserves first-seen casing. If fewer than three keywords remain, candidates are extracted from the transcript ranked by token frequency, with ties broken toward capitalized tokens (e.g., proper nouns). As a last resort, conservative fallback terms are appended.

The guarantee enforces format compliance where emitting three transcript-grounded keywords removes the empty-output and wrong-count failure modes. It does not floor the keyword score, however, which the challenge defines as $K_i = 2(N_{\text{rel}} - N_{\text{irrel}})$ over the three keywords; a summary whose fallback

terms are all judged irrelevant can still reach $K_i = -6$. Since transcript-derived terms inherit the source topic, they score as relevant and keep K_i high in practice, an effect analyzed across model families in Section V.

H. Result Transmission

The `result_transmitter` sink node maps internal keys to the challenge contract (`from`, `to`, `summary`, `keywords`, `proc_time`) and saves each window as `window_i.json`. When `destination_endpoint` is configured, results are also posted via HTTP. A debug mode persists the full transcript and raw LLM output alongside each window for post-hoc analysis during development.

IV. IMPLEMENTATION AND DEPLOYMENT

Pipeline configuration is split across two JSON files to decouple node topology from model-specific parameters. `config-base.json` declares all nodes except the summarizer; a profile from `pipelines/summarizer/` supplies the model path, sampling configuration, and VRAM budget for the chosen summarizer. A merge tool assembles these at launch time, and a single shell script serves as the unified entry point for development, evaluation, and submission.

The submission Docker image builds on the NVIDIA CUDA 12.4 runtime with Python 3.12; model weights for both the ASR model and the summarizer are embedded at build time, eliminating any network dependency at inference. A `docker-compose.yml` file orchestrates Janus and the pipeline container as two services. Table I summarizes all pipeline nodes and their key parameters.

TABLE I: Pipeline nodes with key parameters and scoring targets.

Node	Type	Key parameters	Target
audio_rtp	source	Opus/48k, port 8888	+4
audio_chunker	proc	5s chunk, 1s overlap	—
transcriber	proc	float16, GPU, VAD on	B_i
halluc. filter	proc	22 exact patterns	B_i
novel_extractor	proc	10 words, $\text{sim} \geq 0.8$	B_i
window_agg.	proc	300s	—
summarizer	proc	Qwen3.5-4B-AWQ, $T=0.15$	B_i, L_i
transmitter	sink	JSON + HTTP POST	—

Since the summarizer runs vLLM in-process (Sections II and III-F) rather than as a separate service, the submission ships as a single self-contained container with no inference daemon to orchestrate.

V. EVALUATION

We evaluate the pipeline against the composite challenge score $C_i = B_i + K_i + L_i$, isolating the contribution of each design decision rather than reporting a single aggregate. On the Rev16 podcast evaluation, the submission configuration (Qwen3.5-4B-AWQ summarizer with `large-v3-turbo` ASR) attains a final source score of 40.14 ± 0.85 out of 45, holding K_i at its ceiling on every evaluated window while keeping average processing time under 1 s. In this section, we

detail the measurement methodology, compare ASR models, report per-window scoring, ablate individual components, and sweep summarizer families, before stating threats to validity.

A. Setup

Experiments cover the podcast episodes from the Rev16 dataset [25]. A 300 s window pipeline run produces five scored windows per episode. To absorb LLM-judge non-determinism, every model comparison repeats the pipeline at least twice per configuration. The numbers reported throughout are means across these runs, accompanied by standard deviations where more than one run is available. The offline judge harness uses a cross-family panel of three int4-AWQ judges (Mistral-Small-24B-Instruct, Phi-4, and Gemma-3-27B-IT) chosen from outside the Qwen family to avoid self-evaluation bias. Judges run sequentially in isolated subprocesses; each process exits to reclaim VRAM before the next loads, since the 24 GB development GPU cannot host the full panel simultaneously. Following the G-Eval paradigm [13], each judge scores every summary against the five Likert criteria defined by the challenge and evaluates keyword relevance as a per-keyword boolean flag. Panel consensus is reported as the mean and standard deviation across the three judges. Low dispersion lends confidence to a score, while high dispersion flags judge-specific bias and is treated with caution. All experiments ran on an NVIDIA RTX 4090 with 24 GB of VRAM under CUDA 12.4, while the submission target is the larger RTX Pro 4500 with 32 GB.

Word Error Rate (WER), which is not part of the $B_i/K_i/L_i$ scoring formula, serves as an independent diagnostic and is measured via jiwer against ground-truth transcripts. Since the ground-truth text lacks timestamps, proportional character-based alignment with a sliding-window search ($\pm 30\%$ offsets) locates the best-matching reference segment for each window.

B. ASR Model Comparison

Table II compares two faster-whisper variants under the locked Qwen3.5-4B-AWQ summarizer, with two pipeline runs per configuration scored by the cross-family panel (same Rev16 episodes as Section V-F). The two models differ by 0.7 WER percentage points (0.252 vs. 0.259) and 0.46 points in $B_i + K_i$ (29.30 vs. 29.76), differences within cross-run judge noise. Despite the marginally lower WER of `large-v3-turbo`, `small.en` achieves a slightly higher final score (40.44 vs. 40.14), confirming that transcription accuracy is not the scoring bottleneck at 300 s granularity. The submission pipeline retains `large-v3-turbo` for robustness to non-English challenge audio.

C. End-to-End Scoring

Table III reports per-window scores for Qwen3.5-4B-AWQ on a 25-minute podcast episode (5 windows of 300 s each), averaged across two independent runs to reduce LLM-judge variance. All five windows achieve $K_i = 6.0$ (the maximum), confirming that the keyword guarantee reliably fills all three slots with relevant terms. B_i remains narrowly bracketed

TABLE II: ASR model comparison at the locked Qwen3.5-4B-AWQ summarizer (Rev16 episodes, two pipeline runs per configuration for small.en; the large-v3-turbo numbers reuse the runs collected for Section V-F). Higher is better for $B + K$, L , and the final score; lower for proc_i and WER.

Metric	small.en	large-v3-turbo
Avg $B_i + K_i$	29.76	29.30
Avg L_i	6.68	6.84
Avg proc_i (s)	0.81	0.76
Avg WER	0.259	0.252
Final score (cross-ep mean)	40.44	40.14
Std across runs	0.59	0.82

between 24.0 and 24.7 out of 25, close to the Likert ceiling. Run-to-run variation in C_i never exceeds 0.9 points on any window, attributable to judge non-determinism rather than pipeline instability. The latency component shows the widest spread in the table and ranges from $L_i = 6.05$ on window 3 to $L_i = 7.18$ on window 4. Window 3 spans a dense 300 s segment that took 1.01 s on average to summarize, while window 4 covers only the trailing 105 s of audio and was therefore compressed in 0.66 s. Averaging C_i across the five windows yields $\bar{C} = 37.1$, which, combined with the Janus bonus, produces a final source score of **41.09** for this episode. Cross-domain validation on Internet Engineering Task Force (IETF) 125 sessions (Computing-Aware Traffic Steering, Media over Quick UDP Internet Connections (QUIC)) produced scores of 40.19 and 40.24, closely tracking the Rev16 mean (Section V-F) despite denser technical vocabulary.

TABLE III: Per-window scores, Qwen3.5-4B-AWQ [23], 300 s windows (ep. 27, 2-run mean, cross-family panel consensus). t = avg processing time (s).

Win.	From	To	t (s)	B_i	K_i	L_i	C_i
0	0	300	0.86	24.7	6.0	6.51	37.2
1	300	600	0.72	24.7	6.0	6.99	37.7
2	600	900	0.85	24.0	6.0	6.53	36.5
3	900	1200	1.01	24.3	6.0	6.05	36.4
4	1200	1305	0.66	24.5	6.0	7.18	37.7
avg			0.82	24.4	6.0	6.65	37.1

D. Outlier Analysis

Since B_i and K_i stay flat across the windows of Table III, the residual C_i variation is latency-driven rather than a content-quality failure; the sole outlier risk is a dense-speech window that inflates proc_i . In exploratory shorter-window runs, topic transitions produced B_i drops of 3–6 points and K_i swings as large as -4 , variance that the 300 s aggregation averages out.

E. Ablations

The hallucination filter, novel extractor, and keyword guarantee (Sections III-C, III-D and III-G) each defend a specific scoring term; we assess their effect qualitatively here. The

keyword guarantee is the most consequential. For Qwen3.5-4B-AWQ it holds K_i at the maximum of 6.0 on every evaluated window, and the format compliance it enforces removes the empty-output failure mode outright. It sets no hard floor, however: under the weaker Llama-3.1-8B summarizer, two Ep 11 windows still scored $K_i = -6$ from the Gemma judge, where transcript-derived fallbacks inherited the low-relevance signal of poorly summarized content. Upstream, the hallucination filter and novel extractor protect B_i by suppressing the 22 known artifact phrases and the $\approx 20\%$ overlap redundancy before either can distort the summary.

F. Model-Size and Cross-Family Sweep

Figure 3 decomposes scores for six configurations across Rev16 episodes (300 s windows, at least two runs per configuration). Within the Qwen3.5 family, $B_i + K_i$ is nearly flat across all four variants; score differences are driven primarily by latency, with proc_i ranging from 0.79 s (4B-AWQ) to 2.51 s (9B-BF16). AWQ reduces proc_i by 35 % at 4B and 30 % at 9B, yielding source scores of 40.14 ± 0.85 (4B-AWQ), 38.97 ± 0.91 (4B-BF16), 37.15 ± 1.53 (9B-AWQ), and 36.12 ± 1.42 (9B-BF16). Gemma-3-12B-IT-AWQ [26] scores 35.63 ± 1.06 despite having $3\times$ more parameters than 4B-AWQ; its $B_i + K_i = 28.1$ is only marginally below the Qwen mean of 29.2, but $\text{proc}_i \approx 2.07$ s ($L_i = 3.55$) erases ≈ 3 score points relative to 4B-AWQ, confirming that beyond the quality gate, latency is the binding lever. Llama-3.1-8B-Instruct [27] scores 32.31 ± 4.59 ; episode-to-episode $\bar{B}_i$ swings from ≈ 13 to ≈ 24 , reflecting content-driven instability that the keyword guarantee cannot resolve. Qwen3.5-4B-AWQ is selected as the submission summarizer.

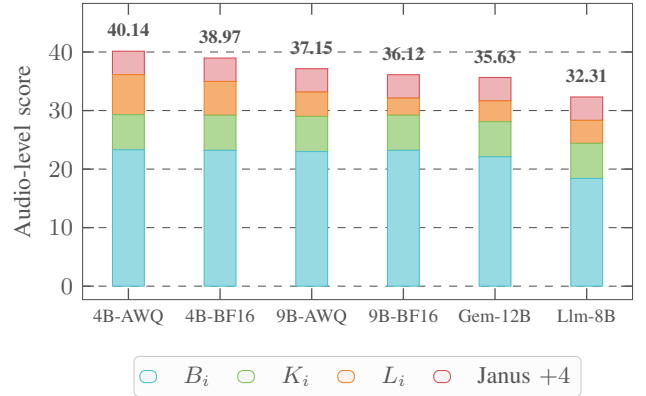


Fig. 3: Cross-audio mean score decomposition across six models (Qwen3.5-4B-AWQ, Qwen3.5-4B-BF16, Qwen3.5-9B-AWQ, Qwen3.5-9B-BF16, Gemma-3-12B-IT-AWQ, Llama-3.1-8B-Instruct). $B_i + K_i$ is flat within the Qwen family; score differences come from L_i .

G. Threats to Validity

Several limitations affect the reported numbers. Our offline judge panel of Mistral-Small-24B, Phi-4, and Gemma-3-27B (all int4-AWQ) serves as a proxy for the hidden challenge

judge, not an exact replica. Our offline judge panel of Mistral-Small-24B, Phi-4, and Gemma-3-27B (all int4-AWQ) is a proxy for the hidden challenge judge, so even where the three agree the reported scores are directional rather than absolute. Repeated runs on identical inputs produce C_i values that differ by 0.1–0.9 points, and content variation across audio episodes contributes a further 0.9–1.5 points of score variance. Averaging across runs reduces judge noise but not content-driven variance. Evaluation covers Rev16 podcast episodes and IETF 125 technical sessions (Section V-C); the held-out challenge audio may differ in speaker count, topic complexity, and acoustic conditions, so its ranking is not guaranteed to match. WER measurement relies on proportional character alignment against untimestamped ground truth, which introduces alignment noise on top of the underlying transcription errors.

VI. DISCUSSION AND CONCLUSION

The scoring formula couples quality and speed asymmetrically in which the latency gate withholds any L_i reward until $B_i \geq 10$, and each irrelevant keyword costs -2 , the value of a 0.5 s latency gain. Our pipeline answers this ordering, securing B_i above the gate and protecting K_i before reducing proc_i .

The resulting eight-node Juturna pipeline summarizes live Opus/RTP audio in real time, averaging 40.14 ± 0.85 out of 45 across Rev16 episodes. Three mechanisms drive this result: (i) streaming chunk deduplication protects B_i , (ii) the keyword guarantee secures the format underlying K_i , and (iii) thinking-mode suppression with int4-AWQ quantization holds proc_i low for L_i . Our pipeline holds B_i at 24–25 and K_i at its ceiling while keeping processing time near 0.79 s, leaving latency as its main remaining lever.

We also explored supervised fine-tuning of the Qwen3.5-4B summarizer on the Rev16 episodes, adapting the base model with QLoRA [28] on a teacher-distilled dataset of judge-filtered summary and keyword targets. Across three variants spanning teacher capacity and label coverage, none improved the final source score over the base model. The strongest, distilled entirely from a high-capacity teacher, lifted B_i from ≈ 19 to ≈ 21 but lengthened the outputs, pushing proc_i from ≈ 0.7 s to ≈ 1.05 s and regressing L_i enough to leave the untuned base ahead. This follows directly from the asymmetric scoring this work targets, where a quality gain that costs processing time is penalized, therefore the submission ships the base summarizer with a tuned prompt.

If concurrency grows or the latency budget tightens, an SGLang backend could reclaim part of that margin without affecting B_i . A second direction hardens robustness, replacing JSON parsing with structured-output decoding and adapting the prompt to topic transitions to curb the B_i variance seen on dense windows.

REFERENCES

- [1] Meetecho, “Janus: An open source, general purpose, WebRTC server,” [Online] Available: <https://github.com/meetecho/janus-gateway>, 2014, Accessed on June 1, 2026.
- [2] —, “Juturna: A python library for fast data pipeline prototyping,” [Online] Available: <https://github.com/meetecho/juturna>, 2024, Accessed on June 1, 2026.
- [3] H. Schulzrinne, S. L. Casner, R. Frederick, and V. Jacobson, “RTP: A transport protocol for real-time applications,” [Online] Available: <https://datatracker.ietf.org/doc/rfc3550>, 2003, accessed on June 1, 2026.
- [4] J.-M. Valin, K. Vos, and T. B. Terriberry, “Definition of the Opus audio codec,” [Online] Available: <https://datatracker.ietf.org/doc/rfc6716>, 2012, accessed on June 1, 2026.
- [5] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *International conference on machine learning*. PMLR, 2023.
- [6] OpenNMT, “CTranslate2,” [Online] Available: <https://github.com/opennmt/ctranslate2>, 2026, Accessed on June 1, 2026.
- [7] G. Klein, “faster-whisper: Faster whisper transcription with CTranslate2,” [Online] Available: <https://github.com/SYSTRAN/faster-whisper>, 2026, accessed on June 1, 2026.
- [8] A. Yang, A. Li *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [9] W. Kwon, Z. Li *et al.*, “Efficient memory management for large language model serving with pagedattention,” in *Proc. SOSP*, 2023.
- [10] L. Zheng, L. Yin *et al.*, “SGLang: Efficient execution of structured language model programs,” in *Proc. NeurIPS*, 2024.
- [11] NVIDIA, “TensorRT-LLM: Llm inference in c/c++,” [Online] Available: <https://github.com/NVIDIA/TensorRT-LLM>, accessed on June 1, 2026.
- [12] G. Gerganov, “llama.cpp: Llm inference in c/c++,” [Online] Available: <https://github.com/ggerganov/llama.cpp>, 2023, accessed on June 1, 2026.
- [13] Y. Liu, D. Iter *et al.*, “G-Eval: NLG evaluation using GPT-4 with better human alignment,” in *Proc. EMNLP*, 2023.
- [14] D. Macháček, R. Dabre, and O. Bojar, “Turning whisper into real-time transcription system,” in *Proc. IJCNLP-AACL System Demonstrations*, 2023.
- [15] S. Saranya, B. Bharathi, S. Gomathy Dhanya *et al.*, “Real-time continuous tamil dialect speech recognition and summarization,” *Circuits, Systems, and Signal Processing*, vol. 44, pp. 2855–2881, 2025.
- [16] K. Matsuura, T. Ashihara, T. Moriya, M. Mimura, T. Kano, A. Ogawa, and M. Delcroix, “Sentence-wise speech summarization: Task, datasets, and end-to-end modeling with LM knowledge distillation,” in *Proc. Interspeech*, 2024.
- [17] S. Ganguly, B. Sadhukhan, S. Mandal, S. Sarkar, N. Das, and S. Paul, “WhisperSum: Unified audio-to-text summarization,” in *Proc. IACIS*, 2024, IEEE doc. 10721926.
- [18] Z. Liu and N. F. Chen, “Dynamic sliding window for meeting summarization,” in *Proc. SummDial Workshop at SIGDial*, 2021.
- [19] S. Asthana, S. Hilleli, P. He, and A. Halfaker, “Summaries, highlights, and action items: Design, implementation and evaluation of an LLM-powered meeting recap system,” in *Proc. ACM CSCW*, 2024.
- [20] F. Schneider, M. Turchi, and A. Waibel, “Policies and evaluation for online meeting summarization,” *arXiv preprint arXiv:2502.03111*, 2025.
- [21] D. Jimenez, “Faster whisper ct2 large-v3 turbo,” <https://huggingface.co/deepdml/faster-whisper-large-v3-turbo-ct2>, 2024, Accessed on June 1, 2026.
- [22] Silero Team, “Silero VAD: pre-trained enterprise-grade voice activity detector (VAD), number detector and language classifier,” [Online] Available: <https://github.com/snakers4/silero-vad>, 2024, Accessed on June 1, 2026.
- [23] cyankiwi, “Qwen3.5-4B-AWQ-BF16-INT4: int4-AWQ quantisation of Qwen3.5-4B,” [Online] Available: <https://huggingface.co/cyankiwi/Qwen3.5-4B-AWQ-BF16-INT4>, 2025, Accessed on June 1, 2026.
- [24] J. Lin, J. Tang *et al.*, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proc. MLSys*, 2024.
- [25] distil-whisper, “Rev16: Long-form podcast transcription evaluation set,” [Online] Available: <https://huggingface.co/datasets/distil-whisper/rev16>, 2024, Accessed on June 1, 2026.
- [26] G. Ernst, “Gemma-3-12B-IT-AWQ: int4-AWQ quantisation of Gemma-3-12B-IT,” [Online] Available: <https://huggingface.co/gaunernst/gemma-3-12b-it-int4-awq>, 2025, Accessed on June 8, 2026.
- [27] Meta AI, “Llama-3.1-8B-Instruct,” [Online] Available: <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>, 2024, Accessed on June 8, 2026.
- [28] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.